DS 4 – Project Report: LLMs + Evaluation

1. Introduction

In this project I used 3 models and approaches to answer 643 Austrian tax law questions. The questions were manually created and have an according answer. These answers were not used in any kind to train the models. These Q&As were only used in the evaluation phase to assess the correctness of the 3 models I created. The first model is an inference model, which accesses the OpenAI via API. Then the questions were answered without further data input or training. The second model is a Retrieval-Augmented Generation model which is based on Mistral 7B in combination with ChromaDB for the Retrieval part. As an input this model got 30 of the relevant paragraphs. The last model uses a Fine-Tuning approach and is based on Qwen 2-1.5B. This model was fine-tuned with 113 questions from various sources across the internet (e.g.: tax advisor websites, WU tax law exam questions etc.). All three models were then evaluated using the same metrics to enable a structured comparison (ROUGE, BERTScore, BLEU).

2. Models

Inference model: OpenAI API (gpt-5.4-mini)

Input: 643 questions (dataset_clean.csv)

Data:

The dataset with the 643 questions was used as an input. No further data was used.

Model:

I used the gpt-5.4-mini via the OpenAI API. This is a smaller model, which can process faster but still delivers very useful output. The question in combination with the prompt is being sent to the model which doesn't run on the local device. The model is accessed via HTTP requests and solely answers from the pre-trained knowledge of the model – I didn't provide any documents etc. There is no specific training or fine tuning done with the training on my side. Just the prompt is modified. Also, temperature is set to 0 – because for legal text I don't need variability – there should only be one solution and that's what the output should be. If I ask the same question again, I want the same output which is in best case the correct one. In the prompt I stated that the answer should be short and precise. I also explained the context with a short sentence that the model should answer questions about Austrian tax law and if possible, it should also cite the used paragraphs.

prompt:

```
                "content": f"""Beantworte die folgende steuerrechtliche Frage für Österreich kurz und präzise.
Gib nur die Antwort, keine Einleitung. Reiner Text: kein Fettgeschriebenes etc..
Wenn möglich, nenne die relevante gesetzliche Grundlage.
Question:
```

RAG model: Mistral 7B-Instruct-v0.3 + ChromaDB

Input: 643 questions (dataset_clean.csv) + extracts of the relevant law (downloaded from the RIS.at and then modified)

Source: https://www.ris.bka.gv.at/

Processing Duration: ca. 120– 180 minutes

Data:

I downloaded the relevant paragraphs from https://www.ris.bka.gv.at as RTF-documents (parts of EStG, KStG, UStG, GrEStG). These documents I then cleaned manually so only the paragraphs with the explanation were left. I also cleaned the formatting so only relevant

information remained. These extracts I then uploaded to use as Data for the RAG-model. Only 30 paragraphs were used to have a small but sufficient dataset.

Model:

In general, my RAG-model consists of 4 steps: load/process the paragraphs, chunking, embedding + ChromaDB and the RAG function.

Load & Process the Data

After the manual preprocessing the 30 relevant paragraphs are transformed from RTF to plain text and the whitespace is normalized.

Chunking

To be able to use the input data/text for the RAG function I need to chunk the text into fragments that can be better processed. Therefore, I used 2 approaches: Split at § symbols in the text (especially for the GrEStG: contains multiple paragraphs) and a split after 300 words (to make big paragraphs also usable (50 words overlap so the context/meaning doesn't get lost after splits)). With these two approaches 637 chunks were created. Due to the presence of 1-3 § references in the introductory text of each RTF file (each containing 1 paragraph) there were many splits that do not add value. But we need this approach for the GrEStG because we must separate the different paragraphs from each other.

Embedding/ChromaDB

Then I used ChromaDB for the embedding of the input I provided. ChromaDB is a database that converts the input text into vectors that can then be accessed and used by the model (in our case Mistral 7B). For each text sequence (chunk) one vector is created and stored in the Data Base. So, for 300 words one vector is created which "understands" the meaning of this chunk. This is a very large number of words. This clearly represents a limitation of my model because due to these big chunks – details get lost. Smaller chunks per topic would improve retrieval precision but would also lead to even longer processing time, possible loss of legal context because of split sentences. For the embedding ChromaDB uses the model all-Mini-L6-v2 which converts each chunk into a 384-dimensional vector by passing the text through 6 layers and averaging the resulting token representations. Similar texts produce similar vectors – this enables the model to find relevant passages even if the exact wording isn't found. This adds another limitation of my model is the standard embedding model all-Mini-L6-v2 of ChromaDB which is meant for English text – so this could lead to potential wrong connections and "interpretation".

RAG function

For the retrieval and text generation Mistral 7B with 7 billion parameters is used. I chose it because I can run it on a free T4 GPU with 4-bit quantization and still delivers good outputs. In the retrieval part – the question is transformed into a vector. This vector is then compared with the 637 vectors, and the 5 most similar chunks are returned. These 5 chunks are then joined to one long text and embedded into a prompt following Mistral's chat template. The model is instructed to answer in 1-2 sentences, solely using the retrieved legal text and to cite the paragraphs. The prompt is then tokenized and truncated to maximal 2048 tokens due to due to the model's context window limit. This limit also hinders the performance of my model because parts of the chunks are being cut of due to these limited tokens. 5 chunks of up to 300 words will often exceed this limit. Mistral then generates the answer with a maximum of 100 new tokens, while always using the most probable token ensuring deterministic and reproducible outputs (do_sample = False) and a repetition penalty of 1.2 which prevents the model from repeating itself (I had problems with this in the beginning due to repetitions in the input data). Finally, the tokens are extracted and decoded back into readable text.

Fine-Tuning model: Qwen2-1.5B (Unsloth + LoRA)

Input: 113 questions with answers (WKO Q&As, tax advisor websites, old WU tax law exams questions + answers)

Source: https://res-wth.at/service/h%C3%A4ufige-fragen

https://www.salzburg.gv.at/ehrenamt/fragen-und-antworten-steuerrecht

https://www.studocu.com/de-at/document/universitat-wien/steuerrecht/falle-steuerrecht/57034172

https://ksw.or.at/berufszugang-berufsanwaerterinnen/fachpruefungen-wtbg-2017/downloads/#c2123

Data:

To provide the model useful input Q&As I searched the internet for tax law questions. I found multiple on the WKO website, tax advisor websites but mainly I used old WU tax law exam questions with the correct answers. These pairs I then uploaded and used it to fine tune the model.

Model:

For this I used a smaller model Qwen2-1.5B due to the limited 15 GB VRAM for the T4 GPU. I also used LoRA for the fine tuning because full fine tuning wouldn't have been possible with the Google Colab environment due to the limited VRAM and too long training time. LoRA freezes the original model weights and only trains small adapter matrices - reducing the number of trainable parameters from 1.5 billion to approximately 1-5 million. I used Unsloth as the training framework, which further reduced memory usage by 50-70% and made the training much faster. To be able to adapt the hyperparameters along the way I defined them in one cell.

```
MAX_SEQ_LENGTH    = 1040.       # maximum number of tokens per training example
NUM_EPOCHS        = 5           # number of full passes through the training dataset
BATCH_SIZE        = 2           # samples processed per GPU step
GRAD_ACCUMULATION = 4           # simulates a larger batch by accumulating gradients over 4 steps
LEARNING_RATE     = 2e-4        # learning rate: controls how much the model weights change per update step


LORA_R       = 16               # scaling factor applied to the LoRA update
LORA_ALPHA   = 32               # acts as regularization to prevent overfitting on the small training set
LORA_DROPOUT = 0.05
```

Hyperparameters

Number of Epochs was set to 5 so the training dataset is passed 5 times. It was raised to 5 in the process after manual inspection of the answers resulting from a setting with 3 full passes through the data (too high would lead to a problem of overfitting – wasn't tested just manually inspected). Batch Size and Grad accumulation was set the way, so it is processable with the GPU. For the learning rate I used the standard for LoRA fine tuning (according to unsloth's guide for LoRA fine tuning: 2e-4).

LoRA

LoRA introduces two small adapter matrices (A and B) per layer with rank 16, replacing the need to update the full weight matrices - reducing the trainable parameters. The LoRA adapter was then attached to the attention layers of the Qwen model resulting in 4.36 million trainable parameters which resembles 0.28% of the total trainable parameters – the remaining weights stayed the same.

Supervised Fine-Tuning

Each training question was formatted in a structured prompt with an instruction for the setting, the question and the reference answer. Training was performed using Supervised Fine-Tuning (SFT) via Unsloth's SFTTrainer for approximately 2 minutes on the T4 GPU. I used a cosine learning rate scheduler with 5 warmup steps: starting from a low learning rate, rising to 2e-4, then decreasing again while following a cosine curve. This allows large learning steps early in training and finer adjustments towards the end. This prevents the model from overwriting previously learned patterns. The training loss decreased from 2.94 to 1.50 over 75 steps, representing successful learning. The lowest training loss was achieved after 65 steps with 1.41. I overtrained the model a little bit with my 75 steps generating a training loss of 1.49 in the end.

Inference

After training, the fine-tuned model was loaded and optimized for inference using Unsloth's FastLanguageModel, utilizing up to 14.5 GB of the available VRAM on the T4 GPU. Unlike the RAG model which used greedy decoding (do_sample=False), for the fine-tuned model I used sampling with a low temperature of 0.3 and top_p=0.9. This was necessary because the smaller Qwen2-1.5B model showed a tendency towards repetitive outputs with greedy decoding. The repetition penalty was set higher (1.4 vs 1.2) for the same reason. These patters where also inspected manually on 20 example questions and then adjusted accordingly.

3. Evaluation results and conclusion

	ROUGE-1	**ROUGE-L**	**BERTScore**	**BLEU**
Inference	0.3329	0.2296	0.7386	9.6853
RAG	0.2258	0.1674	0.7132	5.0969
Fine Tuning	0.1161	0.0750	0.6523	0.2947

The inference model (GPT-4o-mini) achieved the best results across due to its larger size and extensive pre-training data. The RAG model outperformed the fine-tuned model because of the direct access to the relevant legal texts during generation. The fine-tuned model performed worst, which can be attributed to the small training dataset of only 113 examples and the relatively small model size of 1.5B parameters. It is worth noting that ROUGE and BLEU scores are generally low across all models, which is expected for open-ended legal QA tasks where valid answers can be phrased in many ways BERTScore is therefore the most meaningful metric here.

Sources:

https://medium.com/@nermeen.abdelaziz/build-your-first-python-rag-using-chromadb-openai-d711db1abf66

https://unsloth.ai/docs/de/los-gehts/fine-tuning-llms-guide/lora-hyperparameters-guide

https://www.ibm.com/de-de/think/topics/lora

https://medium.com/@dminhk/retrieval-augmented-generation-rag-explained-b1dd89979681

https://www.geeksforgeeks.org/nlp/introduction-to-chromadb/

https://medium.com/@shashankag14/understanding-sampling-techniques-in-large-language-models-llms-dfc28b93f518

https://unsloth.ai/docs/get-started/fine-tuning-llms-guide/lora-hyperparameters-guide

https://medium.com/@pur4v/understanding-llm-evaluation-metrics-bleu-rouge-exact-match-and-bertscore-716487e40bdd

Models:

https://huggingface.co/mistralai/Mistral-7B-v0.1

https://openai.com/de-DE/api/

https://huggingface.co/docs/transformers/model_doc/qwen2

AI Usage:

For parts of this project I used the assistance of Claude (Sonnet 4.6) by Anthropic. I used this AI as support for the implementation of the evaluation scripts, the integration of data sources and the integration of the models from Hugging Face and OpenAI API. All outputs were reviewed and verified by me.